

POD5 损坏文件重建操作手册

环境配置、脚本说明、单文件与批量运行、完整验证

版本 1.0 | 2026-07-18

适用于：POD5 外层 footer/末尾签名损坏，但内部 Reads、Signal、Run Info Arrow IPC 表仍完整的恢复场景。

本工具为实验性、尽力而为的数据恢复辅助程序，不是 Oxford Nanopore Technologies 官方工具。

目录

1. 工具包范围与目录结构
2. 环境建立与版本锁定
3. 数据与目录准备
4. 单文件标准恢复流程
5. 批量重建流程
6. Run Info donor 使用条件
7. 结果验证与审计材料
8. 常见错误与处理
9. 数据安全规则
10. 工作原理与适用边界
11. 官方参考资料

本工具包用于一种特定损坏模式：POD5 外层 footer、末尾签名或索引损坏，但内部完整的 Reads、Signal、Run Info Arrow IPC 表仍能被提取和读取。工具会丢弃无法完整核对或解压的 read，并使用 POD5 Writer 重新生成合法 POD5。

重要：这是针对数据恢复场景编写的实验性工具，不是 Oxford Nanopore Technologies 官方恢复程序。它不能恢复已被覆盖、TRIM 或恢复为零的字节，也不能重建已经截断的 Arrow record batch。

目录结构

```
POD5_salvage_toolkit/
├── README.md
├── POD5 损坏文件重建操作手册.docx
├── env/
│   ├── environment.yml
│   └── requirements.in
├── examples/
│   └── single_file_workflow.sh
└── scripts/
    ├── batch_rebuild_pod5.sh
    ├── capture_environment.sh
    ├── rebuild_pod5_from_sections_v2.py
    └── validate_rebuilt_pod5.py
```

适用条件

适用：

- 文件开头仍有 POD5 signature；
- `--inspect-only` 能找到完整 Reads 和 Signal 表；
- Run Info 表存在，或有同一次 acquisition 的完整 POD5 作为 donor；
- Signal 数据仍能通过 VBZ 解压；
- 接受跳过少量损坏 read。

不适用：

- 文件主体大量被置零或覆盖；
- Reads/Signal Arrow 文件自身 footer 或 record batch 已截断；
- 只能找到 Signal，找不到 Reads；
- RAID 或文件系统仍在被写入；
- 希望恢复缺失信号或保证与删除前文件逐字节一致。

一、建立环境

方法 A : Python venv (推荐)

```
python3 -m venv ~/envs/pod5-salvage
source ~/envs/pod5-salvage/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r env/requirements.in
```

检查：

```
python - <<'PY'
import sys
import numpy
import pod5
import pyarrow
from pod5.pod5_types import ShiftScalePair

print('python:', sys.version)
print('pod5:', getattr(pod5, '__version__', 'unknown'))
print('pyarrow:', pyarrow.__version__)
print('numpy:', numpy.__version__)
print('ShiftScalePair:', ShiftScalePair(shift=1.0, scale=2.0))
PY
```

方法 B : Conda

```
conda env create -f env/environment.yml
conda activate pod5-salvage
```

成功测试后立即锁定精确版本：

```
scripts/capture_environment.sh environment_record
```

或：

```
python -m pip freeze > requirements.lock.txt
```

以后可用：

```
python3 -m venv ~/envs/pod5-salvage-locked
source ~/envs/pod5-salvage-locked/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.lock.txt
```

二、准备目录

源文件、输出文件、工作目录必须分开。不要覆盖原始损坏文件。

```
mkdir -p /recovery/pod5_input
mkdir -p /recovery/pod5_output
mkdir -p /recovery/pod5_work
```

建议可用空间至少为待处理 POD5 总体积的 2–3 倍，因为工作目录会保存提取的 Arrow 表，同时还要写出新 POD5。

三、单文件标准流程

设置路径：

```
T00LKIT=/path/to/POD5_salvage_toolkit
INPUT=/recovery/pod5_input/PBM24760_0be05009_97897385_0.pod5
BASE=$(basename "$INPUT" .pod5)
```

1. 只检查内部表

```
python "$T00LKIT/scripts/rebuild_pod5_from_sections_v2.py" \
"$INPUT" \
"/recovery/pod5_output/${BASE}.placeholder.pod5" \
```

```
--work-dir "/recovery/pod5_work/${BASE}.inspect.sections" \
--inspect-only \
2>&l | tee "/recovery/pod5_work/${BASE}.inspect.log"
```

理想输出：

```
section=1 type=signal ...
section=2 type=run_info ...
section=3 type=reads ...
reads_table=...
signal_table=...
run_info_table=...
```

Reads 行数和 Signal 行数不必相等，一条 read 可以引用多个 Signal chunk。

2. 重建 100 条测试

```
python "$T00LKIT/scripts/rebuild_pod5_from_sections_v2.py" \
"$INPUT" \
"/recovery/pod5_output/${BASE}.test100.pod5" \
--work-dir "/recovery/pod5_work/${BASE}.test100.sections" \
--max-reads 100 \
--validate-signal \
2>&l | tee "/recovery/pod5_work/${BASE}.test100.rebuild.log"
```

期望：

```
recovered_reads=100
skipped_reads=0
```

少量 skipped 不一定代表整体失败，查看：

```
column -t -s $'\t' \
"/recovery/pod5_work/${BASE}.test100.sections/skipped_reads.tsv" | head
```

3. 完整读取并解压测试 POD5

```
python "$T00LKIT/scripts/validate_rebuilt_pod5.py" \
"/recovery/pod5_output/${BASE}.test100.pod5" \
--expected-reads 100 \
--json "/recovery/pod5_work/${BASE}.test100.validation.json"
```

报告中应为：

```
"reported_reads": 100,
"decoded_reads": 100,
"passed": true
```

4. 用 Dorado 进行真实 basecalling 测试

建议使用完整模型路径，避免自动 chemistry 选择掩盖其他问题。

```
DORADO=/home/prom/software/dorado-2.1.0-linux-x64/bin/dorado
MODEL=/home/prom/software/dorado_models/dna_r10.4.1_e8.2_400bps_hac@v5.2.0

"$DORADO" basecaller \
"$MODEL" \
"/recovery/pod5_output/${BASE}.test100.pod5" \
--emit-fastq \
--device cuda:0 \
```

```
> "/recovery/pod5_output/${BASE}.test100.fastq" \
2> "/recovery/pod5_work/${BASE}.test100.dorado.log"
```

检查退出码和 FASTQ 数量：

```
echo $?
awk 'END {
    if (NR % 4 != 0) {print "FASTQ_FORMAT_ERROR"; exit 1}
    print "reads=" NR/4
}' "/recovery/pod5_output/${BASE}.test100.fastq"
```

5. 全量重建

```
python "$T00LKIT/scripts/rebuild_pod5_from_sections_v2.py" \
"$INPUT" \
"/recovery/pod5_output/${BASE}.rebuilt.pod5" \
--work-dir "/recovery/pod5_work/${BASE}.full.sections" \
--validate-signal \
2>&1 | tee "/recovery/pod5_work/${BASE}.full.rebuild.log"
```

6. 全量验证

```
pod5 inspect summary "/recovery/pod5_output/${BASE}.rebuilt.pod5"

python "$T00LKIT/scripts/validate_rebuilt_pod5.py" \
"/recovery/pod5_output/${BASE}.rebuilt.pod5" \
--json "/recovery/pod5_work/${BASE}.full.validation.json"

sha256sum "/recovery/pod5_output/${BASE}.rebuilt.pod5" \
> "/recovery/pod5_output/${BASE}.rebuilt.pod5.sha256"
```

macOS 可使用：

```
shasum -a 256 file.pod5
```

四、批量重建

先抽样验证至少 1-3 个文件，再批量运行：

```
"$T00LKIT/scripts/batch_rebuild_pod5.sh" \
/recovery/pod5_input \
/recovery/pod5_output \
/recovery/pod5_work
```

批量脚本默认顺序处理，避免并行读写压垮恢复盘或网络存储。结果汇总：

```
column -t -s $'\t' /recovery/pod5_work/batch_summary.tsv | less -S
```

状态含义：

- PASS：重建完成且全部 signal 可完整读取；
- FAILED：重建脚本失败或没有生成有效输出；
- VALIDATION_FAILED：生成了 POD5，但完整 signal 读取验证失败；
- SKIPPED_OUTPUT_EXISTS：目标文件已存在，为防止覆盖而跳过。

五、缺少 Run Info 时使用 donor

仅当 Reads 和 Signal 完整、Run Info 缺失时使用：

```
python "$T00LKIT/scripts/rebuild_pod5_from_sections_v2.py" \
  damaged.pod5 rebuilt.pod5 \
  --work-dir damaged.sections \
  --donor-pod5 intact_same_acquisition.pod5 \
  --validate-signal
```

donor 必须来自同一次 acquisition。不要仅凭芯片编号或样品名判断；应核对 acquisition ID、flow cell、kit、sample rate 和 protocol run ID。

六、输出文件和审计材料

每个文件建议至少保留：

```
原始损坏 POD5（只读保存）
重建 POD5
section_report.json
skipped_reads.tsv
*.rebuild.log
*.validation.json
*.pod5.sha256
requirements.lock.txt
```

只有旧的原始 SHA-256 才能证明文件与删除前完全一致。新生成的 SHA-256 只能作为重建文件今后的完整性基线。

七、常见故障

`Invalid signature in file`

外层 POD5 仍损坏，或使用了原始损坏文件而不是 `.rebuilt.pod5`。

`No complete reads could be reconstructed`

查看 `skipped_reads.tsv`。若所有行都是同一 Python 异常，优先考虑脚本/API 兼容问题；若是 signal index、read ID 或 VBZ 解压错误，则是数据层问题。

`module 'pod5' has no attribute 'ShiftScalePair`

必须使用本工具包的 v2 脚本。它从下面的位置导入：

```
from pod5.pod5_types import ShiftScalePair
```

找不到完整 Reads 或 Signal 表

本脚本不能修复已经截断的 Arrow record batch。需要从其他恢复候选重新取文件，或进行更底层的块级/Arrow IPC carving。

`skipped\_reads` 很高

汇总原因：

```
awk -F '\t' 'NR>1 {sub(/^[^\\t]*\\t/, ""); n[$0]++}
END {for (x in n) print n[x], x}' skipped_reads.tsv | sort -nr | head
```

Dorado 不能自动选择模型

使用完整模型路径，而不是 hac@v... 模型复合名称；同时核对重建后的 Run Info。

八、数据安全规则

1. 永远不覆盖原始损坏 POD5。
2. 不在原 RAID、原恢复卷或仍需取证的设备上写输出。
3. 不把输出目录放在批量输入目录内部。
4. 不删除工作目录，直到 POD5 完整读取和 Dorado 测试都通过。
5. 批量运行前先测试 100 reads。
6. 不把“容器可打开”当作“所有 signal 完整”，必须逐 read 读取 read.signal。
7. 已恢复成零或已被覆盖的信号无法通过本脚本推算回来。

九、原理概要

POD5 基于 Apache Arrow。完整 POD5 包含 Reads、Signal 和 Run Info 表。损坏文件在外层 footer 或末尾签名丢失时，官方 Reader 和 Dorado 会拒绝打开，但内部 Arrow IPC 文件可能仍然完整。本脚本：

8. 读取 POD5 开头的 signature 和 16-byte section marker；
9. 定位并提取 marker 分隔的 Arrow IPC 文件；
10. 按 schema 识别 Reads、Signal、Run Info；
11. 按 Reads 表的 signal 行号关联 Signal 表；
12. 校验 signal read ID、样本数和 VBZ 解压；
13. 跳过损坏 read；
14. 用 pod5.Writer 重新写出 footer、索引和末尾签名。

十、参考资料

- Oxford Nanopore Technologies, POD5 Install: <https://software-docs.nanoporetech.com/pod5/latest/install/>
- Oxford Nanopore Technologies, POD5 Specification: <https://software-docs.nanoporetech.com/pod5/latest/specification/>
- Oxford Nanopore Technologies, Dorado Simplex Basecalling: <https://software-docs.nanoporetech.com/dorado/latest/basecaller/simplex/>
- Oxford Nanopore Technologies, Dorado Model Selection: <https://software-docs.nanoporetech.com/dorado/latest/models/selection/>